

Auto-Tagging EC2 Instances on Launch

Tuesday, July 14, 2026 12:09 PM

Auto-Tagging EC2 Instances on Launch

Objective: Automatically tag newly launched EC2 instances for resource tracking, ownership, and cost allocation.

Instructions:

1. *Lambda IAM Role: Inline policy with `ec2:CreateTags` and `ec2:DescribeInstances`.*
 2. *Lambda Function (Boto3):*
 1. *Extract the instance ID from the EventBridge event (`detail.instance-id`).*
 2. *Tag the instance with `LaunchDate=<current date>` and a custom tag (e.g., `Owner` or `Environment`).*
 3. *Print a confirmation message.*
 3. *EventBridge Rule: Create a rule matching event pattern — source `aws.ec2`, detail-type `EC2 Instance State-change Notification`, state `running` — with the Lambda as target.*
 4. *Testing: Launch a new instance; after a short delay, confirm the tags appear.*
 5. *Bonus: Extract the launching IAM user from CloudTrail events and add an `Owner` tag automatically — this is a popular interview scenario.*
-

Architecture:

EC2 Instance Launch → State changes to "running"



EventBridge Rule (pattern match: `aws.ec2 / running`)



Lambda Function (boto3) → `ec2:CreateTags`



Instance tagged: `LaunchDate`, `Environment`, `Owner` (via CloudTrail lookup)

Step 1: Create the IAM Role for Lambda

Console Navigation

1. Search **IAM** in the top search bar → open it.
2. Left sidebar → **Roles** → **Create role**.
3. Trusted entity type: **AWS service**.
4. Use case: search and select **Lambda** → **Next**.
5. Skip attaching managed policies → **Next**.
6. Role name: `ec2-autotag-lambda-role` → **Create role**.
7. Click into the new role → **Add permissions** → **Create inline policy**.
8. Switch to the **JSON** tab, paste the policy below.
9. **Next** → Policy name: `ec2-autotag-inline-policy` → **Create policy**.

Role Details

Role name
Enter a meaningful name to identify this role.

Maximum 64 characters. Use alphanumeric and "+,=,_,@,-" characters.

Description
Add a short explanation for this role.

Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: "_+=@-/[!#\$%&'()*;,<.>,"

Step 1: Select trusted entities Edit

Trust policy

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "allow",
6       "Action": [
7         "sts:AssumeRole"
8       ],
9       "Principal": {
10        "Service": [
11          "lambda.amazonaws.com"
12        ]
13      }
14    ]
15  }
16 }
```

Step 2: Add permissions Edit

No permissions added

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

- Roles**
- Policies
- IAM users
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

Access reports

- Access Analyzer
- Resource analysis
- Unused access
- Analyzer settings
- Credential report
- Organization activity
- Service control policies
- Resource control policies

Role ec2-autotag-lambda-role created.

Roles (18) Info ⌵

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Search

☐	Role name	Trusted entities	Last activity
☐	AWSServiceRoleForElasticLoadBalancing	AWS Service: elasticloadbalancing	10 days ago
☐	AWSServiceRoleForResourceExplorer	AWS Service: resource-explorer-2	42 minutes ago
☐	AWSServiceRoleForSupport	AWS Service: support (Service-Link)	-
☐	AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service-Link)	-
☐	demofunction-role-rw7wnk3v	AWS Service: lambda	-
☐	ec2-autotag-lambda-role	AWS Service: lambda	-
☐	Lambda-Demo-role-c2ox3umo	AWS Service: lambda	22 days ago
☐	layer-lambda-demo-role-fu6mankq	AWS Service: lambda	22 days ago
☐	new_function-role-bf3fr09	AWS Service: lambda	18 days ago
☐	new_function-role-jnlwze5	AWS Service: lambda	19 days ago
☐	new_function-role-wfwyjaut	AWS Service: lambda	21 days ago

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

- Roles
- Policies
- IAM users
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

Access reports

- Access Analyzer
 - Resource analysis
 - Unused access
 - Analyzer settings
- Credential report
- Organization activity
- Service control policies
- Resource control policies

Related console

- IAM Identity Center
- AWS Organizations

ec2-autotag-lambda-role

Allows Lambda functions to call AWS services on your behalf.

Summary

Creation date

July 14, 2026, 12:38 (UTC+05:30)

Last activity-

ARN

arn:aws:iam::675134942081:role/ec2-autotag-lambda-role

Maximum session duration

1 hour

PermissionsTrust relationshipsTagsLast AccessedRevoke sessions

Permissions policies (0)

You can attach up to 10 managed policies.

Search

Filter by Type

All types

Policy name

Type

Attached entities

No resources to display

SimulateRemoveAdd permissions

Permissions boundary (not set)

Generate policy based on CloudTrail events

You can generate a new policy based on the access activity for this role, then customize, create, and attach it to this role. AWS uses your CloudTrail events to identify the services and actions used and generate a policy. [Learn more](#)

Generate policy

No requests to generate a policy in the past 7 days.

IAM > Roles > ec2-autotag-lambda-role > Create policy

0

Step 1Specify permissions

Step 2Review and create

Review and create

Review the permissions, specify details, and tags.

Policy details

Policy name

Enter a meaningful name to identify this policy.

ec2-autotag-inline-policy

Maximum 128 characters. Use alphanumeric and "+, @, _" characters.

Permissions defined in this policy

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, group, or role), attach a policy to it

Search

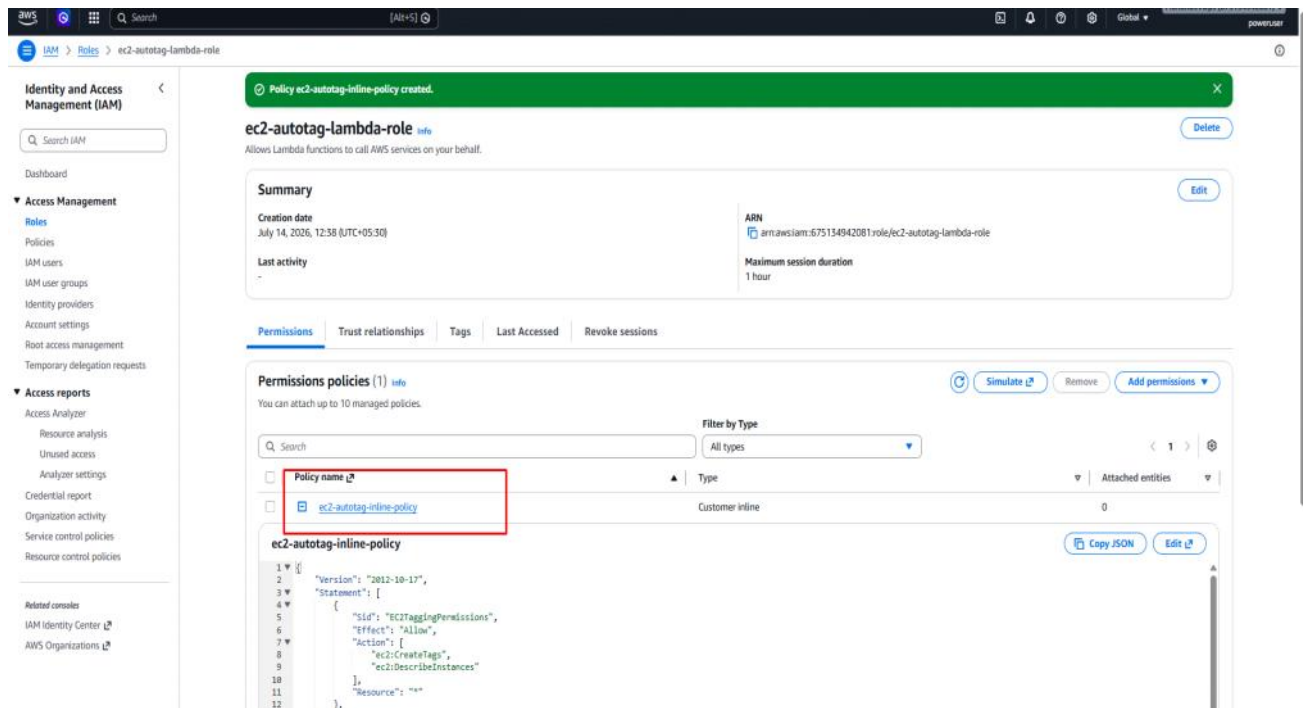
Allow (3 of 472 services)

Show remaining 469 services

Service	Access level	Resource	Request condition
CloudTrail	Limited: Read	All resources	None
CloudWatch Logs	Limited: Write	region[*] string like [All	None
EC2	Limited: List, Tagging	All resources	None

CancelPreviousCreate policy

Auto-Tagging EC2 Instances on Launch Page 3



Grab the Role ARN for the Lambda creation step:

aws iam get-role --role-name ec2-autotag-lambda-role --query 'Role.Arn' --output text

Output :

arn:aws:iam::675134942081:role/ec2-autotag-lambda-role

```
PS D:\Cloud-Devs\Devs-Learn-Projects\Devs-Projects\task3\auto-tagging-EC2-Inst> aws iam get-role --role-name ec2-autotag-lambda-role --query 'Role.Arn' --output text
arn:aws:iam::675134942081:role/ec2-autotag-lambda-role
```

```
PS D:\Cloud-Devs\Devs-Learn-Projects\Devs-Projects\task3\auto-tagging-EC2-Inst>
```

Step 2: Create the Lambda Function

Console Navigation

1. Search **Lambda** → **Functions** → **Create function**.
2. Choose **Author from scratch**.
3. Function name: **ec2-autotag-on-launch**.
4. Runtime: **Python 3.12**.
5. Permissions → **Use an existing role** → select **ec2-autotag-lambda-role**.
6. **Create function**.
7. In the **Code** tab, click **lambda_function.py** in the file tree, select all (Ctrl+A), delete, and paste the full code below.
8. Click **Deploy**.
9. Go to **Configuration** → **General configuration** → **Edit** → set **Timeout** to 30 sec → **Save**.

aws

Search

[Alt+S]

United States (N. Virginia)

Lambda

Functions

Create function

Create function info

Choose one of the following options to create your function.

☒ **Author from scratch**
Start with a simple Hello World example.

☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime info
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Python 3.12

Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose **Custom execution role** under **Additional settings**.

Custom settings info

Durable execution - new
Build resilient, stateful applications with automatic failure recovery.
☐

EC2 capacity provider - new
Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default).
☐

Additional settings
Customize your function architecture, execution role, HTTP(S) endpoints, tenant isolation mode, VPC, code signing, KMS key, and tags.

Additional settings
Customize your function architecture, execution role, HTTP(S) endpoints, tenant isolation mode, VPC, code signing, KMS key, and tags.

General info

ARM64 architecture
Use ARM64 instead of x86_64 (default).
☒

Custom execution role
Use custom execution role instead of new role with basic Lambda permissions (default).
IAM permissions
[ec2-autotag-lambda-role](#)

Networking info

Function URL
Assign HTTP(S) endpoints.
☐

Tenant isolation mode - new
Guarantee separate execution environments for each tenant invoking the function.
☐

VPC
Connect to a VPC to securely access private resources.
☐

Auto-Tagging EC2 Instances on Launch Page 5

The top screenshot shows the AWS Lambda console 'Configuration' tab for a function named 'ec2-autotag-on-launch'. The 'General configuration' section is highlighted, showing a 'Timeout' of '0 min 30 sec' which is enclosed in a red box. Other settings include 'Memory' at 128 MB, 'Ephemeral storage' at 512 MB, and 'SnapStart' set to 'None'. The bottom screenshot shows the VS Code editor with the 'lambda_function.py' file open. The code imports boto3, datetime, and json, and defines a function 'get_launching_lambda_user' that uses boto3 clients for ec2 and cloudtrail to look up events. The status bar at the bottom indicates 'Status: Succeeded' and 'Test Event Name: manualTest'.

Step 3: Create the EventBridge Rule

Console Navigation

1. Search **EventBridge** → **Rules** → **Create rule**.
2. Name: **ec2-autotag-on-running-rule**. Event bus: **default**. Rule type: **Rule with an event pattern** → **Next**.
3. Event source: **AWS events or EventBridge partner events**.
4. Under **Event pattern**, click **Custom pattern (JSON editor)** and paste the pattern below.
5. **Next** → Target type: **AWS service** → **Select a target: Lambda function** → choose **ec2-autotag-on-launch** → **Next** → **Next** → **Create rule**.

Event Pattern: **event-pattern.json**

```
{
  "source": ["aws.ec2"],
  "detail-type": ["EC2 Instance State-change Notification"],
  "detail": { "state": ["running"] }
}
```

Creating the target this way also auto-adds the required `lambda:InvokeFunction` resource-based permission for EventBridge. If you create the rule purely via CLI, you must add that permission yourself (see below)

The screenshot displays the Amazon EventBridge console. The left sidebar shows the navigation menu with 'Rules' highlighted under the 'Buses' section. The main content area shows the 'Rules' page with a 'Create rule' button highlighted with a red box. The table below shows a single rule named 'ec2-autotag-on-running-rule'.

Name	Status	Type	Event bus	ARN	Description
ec2-autotag-on-running-rule	Enabled	Standard	default	arn:aws:events:us-east-1:6751349:42081rule/ec2-autotag-on-running-rule	

Amazon EventBridge > Rules > ec2-autotag-on-running-rule

Amazon EventBridge <

Dashboard

▼ Developer resources

Learn

Sandbox

Quick starts

▼ Buses

Event buses

Rules Updated

Global endpoints

Archives

Replays

▼ Pipes

Pipes

▼ Scheduler

Schedules

Schedule groups

Scheduled rules (legacy) Updated

▼ Integration

Partner event sources

ec2-autotag-on-running-rule

Edit Disable Delete CloudFormation Template

Rule details info

Rule name: ec2-autotag-on-running-rule

Status: Enabled

Event bus name: default

Type: Standard

Description: -

Rule ARN: arn:aws:events:us-east-1:675134942081:rule/ec2-autotag-on-running-rule

Event bus ARN: arn:aws:events:us-east-1:675134942081:event-bus/default

Event pattern Targets Monitoring Tags

Event pattern info

```

1 {
2   "source": ["aws.ec2"],
3   "detail-type": ["EC2 Instance State-change Notification"],
4   "detail": {
5     "state": ["running"]
6   }
7 }

```

Copy

Amazon EventBridge > Rules > ec2-autotag-on-running-rule

Amazon EventBridge <

Dashboard

▼ Developer resources

Learn

Sandbox

Quick starts

▼ Buses

Event buses

Rules Updated

Global endpoints

Archives

Replays

▼ Pipes

Pipes

▼ Scheduler

Schedules

Schedule groups

Scheduled rules (legacy) Updated

▼ Integration

Partner event sources

ec2-autotag-on-running-rule

Edit Disable Delete CloudFormation Template

Rule details info

Rule name: ec2-autotag-on-running-rule

Status: Enabled

Event bus name: default

Type: Standard

Description: -

Rule ARN: arn:aws:events:us-east-1:675134942081:rule/ec2-autotag-on-running-rule

Event bus ARN: arn:aws:events:us-east-1:675134942081:event-bus/default

Event pattern Targets Monitoring Tags

Targets

Details	Target Name	Type	ARN	Input	Role
▼	ec2-autotag-on-launch	Lambda function	arn:aws:lambda:us-east-1:675134942081:function:ec2-autotag-on-launch	Matched event	Amazon_EventBridge_Invoke_Lambda_635542110

Input to target: Matched event

Additional parameters: --

Dead-letter queue (DLQ): -

Step 4: Testing

4.1 Launch a Test Instance

Console:

1. **EC2 → Launch instance** → pick any free-tier AMI (e.g., Amazon Linux 2023) and t2.micro/t3.micro.
2. Launch with default settings.
3. Wait 30–90 seconds after the instance reaches **running**,
4. then check:

Console: **EC2 → Instances** → select the instance → **Tags** tab.

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

▼ **Application and OS Images (Amazon Machine Image)** [info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Search an instance

64-bit (x86)  0000-0000-0000-0000 0000-0000 0000-0000 0000-0000 

▼ Key pair (login) [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

trvl-mem-tst

▼

 [Create new key pair](#)

EC2 > Instances > i-0d7697d521207f6d1

Instance summary for i-0d7697d521207f6d1 (Ec2)

Updated less than a minute ago

Instance ID: i-0d7697d521207f6d1

IPv6 address: -

Hostname type: IP name: ip-10-0-10-137.ec2.internal

Answer private resource DNS name: -

Auto-assigned IP address: 98.80.204.11 [Public IP]

IAM role: -

IMDSv2: Required

Operator: -

Public IPv4 address: 98.80.204.11 | [open address](#)

Instance state: **Running**

Private IP DNS name (IPv4 only): ip-10-0-10-137.ec2.internal

Instance type: t3.micro

VPC ID: vpc-0f6b993a5304573b (brav-mem-vpc) | [open address](#)

Subnet ID: subnet-04953f1fbc97206d (brav-mem-vpc-subnet-public1-us-east-1a) | [open address](#)

Instance ARN: arn:aws:ec2:us-east-1:675134942081:instance/i-0d7697d521207f6d1

Private IPv4 addresses: 10.0.10.137

Public DNS: ec2-98-80-204-11.compute-1.amazonaws.com | [open address](#)

Elastic IP addresses: -

AWS Compute Optimizer finding: [Opt-in to AWS Compute Optimizer for recommendations. | Learn more](#)

Auto Scaling Group name: -

Managed: false

Tags

AMI ID: ami-0b6d9c3d33ba97d99

AMI name: ubuntu/images/hvm-ssd-gp3/ubuntu-resolute-26.04-ami64-server-20260604

Stop protection: Disabled

Instance reboot migration: Default (On)

Monitoring: disabled

Allowed image: -

Launch time: Tue Jul 14 2026 16:09:01 GMT+0530 (India Standard Time) (about 1 hour)

Instance auto-recovery: Default

Platform details: Linux/UNIX

Termination protection: Disabled

AMI location: amazon/ubuntu/images/hvm-ssd-gp3/ubuntu-resolute-26.04-ami64-server-20260604

Lifecycle: normal

Tags

Key | Value

Name | Ec2

Owner | arn:aws:iam::675134942081:user/****

LaunchDate | 2026-07-14

Environment | Dev

Verify Lambda Execution

Console: **CloudWatch** → **Log groups** → **/aws/lambda/ec2-autotag-on-launch** → open latest log stream → confirm SUCCESS: Tagged instance

Log file:

2026-07-14T11:39:15.330000+00:00 2026/07/14/[\$LATEST]61c9a3b37e82480dbe1e27a56feb68f8

Received event:

```
{ "version": "0", "id": "3b8d0c78-01ec-89d1-ae5b-c3b6ec52e715", "detail-type": "EC2 Instance State-change Notification", "source": "aws.ec2", "account": "675134942081", "time": "2026-07-14T10:39:21Z", "region": "us-east-1", "resources": [ "arn:aws:ec2:us-east-1:675134942081:instance/i-0d7697d521207f6d1" ], "detail": { "instance-id": "i-0d7697d521207f6d1", "state": "running" } }
```

2026-07-14T11:39:16.266000+00:00 2026/07/14/[\$LATEST]61c9a3b37e82480dbe1e27a56feb68f8

SUCCESS: Tagged instance i-0d7697d521207f6d1 with LaunchDate=2026-07-14, Environment=Dev, Owner=arn:aws:iam::675134942*:user/******

```
2026-07-14T18:39:21.713Z      START RequestId: 584138b2-115d-40ea-a50c-1bd7085aa0ed Version: $LATEST

2026-07-14T18:39:21.726Z      Received event: {"version": "0", "id": "3b0d0c78-01ec-89d1-ae5b-c3b6ec52e715", "detail-type": "EC2 Instance State-change Notification", "source": "aws.ec2", "account": "675134942081", "time": "2026-07-14T18:39:21Z", "region": "us-east-1", "resources": [{"arn": "arn:aws:ec2:us-east-1:675134942081:instance/i-0d7697d521287f6d1"}]}

Received event:
{
  "version": "0",
  "id": "3b0d0c78-01ec-89d1-ae5b-c3b6ec52e715",
  "detail-type": "EC2 Instance State-change Notification",
  "source": "aws.ec2",
  "account": "675134942081",
  "time": "2026-07-14T18:39:21Z",
  "region": "us-east-1",
  "resources": [
    "arn:aws:ec2:us-east-1:675134942081:instance/i-0d7697d521287f6d1"
  ],
  "detail": {
    "instance-id": "i-0d7697d521287f6d1",
    "state": "running"
  }
}

2026-07-14T18:39:22.825Z      SUCCESS: Tagged instance i-0d7697d521287f6d1 with LaunchDate=2026-07-14, Environment=Dev, Owner=arn:aws:iam:675134942081:user/poweruser

SUCCESS: Tagged instance i-0d7697d521287f6d1 with LaunchDate=2026-07-14, Environment=Dev, Owner=arn:aws:iam:675134942081:user/poweruser
```

Manual Test with a Synthetic Event (console Test tab)

Go to your function → **Test** tab → **Create new event** → use this payload instead of the default {}:

```
{
  "version": "0",
  "id": "test-event-id",
  "detail-type": "EC2 Instance State-change Notification",
  "source": "aws.ec2",
  "account": "123456789012",
  "time": "2026-07-14T10:00:00Z",
  "region": "us-east-1",
  "resources": ["arn:aws:ec2:us-east-1:123456789012:instance/i-0123456789abcdef0"],
  "detail": {
    "instance-id": "i-0123456789abcdef0",
    "state": "running"
  }
}
```

Test event info

Delete

CloudWatch Logs Live Tail

Save

Test

To invoke your function without saving an event, modify the event, then choose Test. Lambda uses the modified event to invoke your function, but does not overwrite the original event until you choose Save.

Test event action

Create new event

Edit saved event

Invocation type

Synchronous

Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

Asynchronous

Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

Event name

manualTest

Event JSON

Format JSON

```

1 {
2   "version": "0",
3   "id": "3b8d8c78-01ec-89d1-ae5b-c3b6ec52e715",
4   "detail-type": "EC2 Instance State-change Notification",
5   "source": "aws.ec2",
6   "account": "675134942081",
7   "time": "2026-07-14T10:39:21Z",
8   "region": "us-east-1",
9   "resources": [
10    "arn:aws:ec2:us-east-1:675134942081:instance/i-0d7697d521207f6d1"
11  ],
12  "detail": {
13    "instance-id": "i-0d7697d521207f6d1",
14    "state": "running"
15  }
16 }

```

1:1 JSON

Info

Choose a

Learn how t cases in AWS

Create i

In this tu to:

- Build consis functi that o
- Invoke its fur

Learn mo

Start

Code

Test

Monitor

Configuration

Aliases

Versions

Executing function: succeeded [logs](#)

Details

```
{
  "statusCode": 200,
  "body": "Tagged i-0d7697d521207f6d1 successfully"
}
```

Summary

Code SHA-256

LQvL6I5MxSvy95POovd6/srOwyFJQ/GUmhh5zYUPk2U~

Execution time

12 seconds ago

Function version

\$LATEST

Request ID

e20be9bb-795b-473a-9ff4-2bd4c9cd54a

Duration

606.63 ms

Billed duration

607 ms

Resources configured

128 MB

Max memory used

100 MB

Log output

The area below shows the last 4 KB of the execution log. [Click here](#) to view the corresponding CloudWatch log group.

```

START RequestId: e20be9bb-795b-473a-9ff4-2bd4c9cd54a Version: $LATEST
Received event: {"version": "0", "id": "3b8d8c78-01ec-89d1-ae5b-c3b6ec52e715", "detail-type": "EC2 Instance State-change Notification", "source": "aws.ec2", "account": "675134942081", "time": "2026-07-14T10:39:21Z", "region": "us-east-1", "resources": ["arn:aws:ec2:us-east-1:675134942081:instance/i-0d7697d521207f6d1"], "detail": [{"instance-id": "i-0d7697d521207f6d1", "state": "running"}]}
SUCCESS: Tagged instance i-0d7697d521207f6d1 with LaunchDate=2026-07-14, Environment=Dev, Owner=arn:aws:iam::675134942081:user/██████████
END RequestId: e20be9bb-795b-473a-9ff4-2bd4c9cd54a
REPORT RequestId: e20be9bb-795b-473a-9ff4-2bd4c9cd54a Duration: 606.63 ms Billed Duration: 607 ms Memory Size: 128 MB Max Memory Used: 100 MB

```